

Universitat d'Alacant
Universidad de Alicante

Escuela Politécnica Superior

Departamento de Ciencia de la Computación e Inteligencia Artificial

DOCUMENTACIÓN DE LA PRÁCTICA

Práctica 1: Sistemas Expertos

Alumno: Kacper Piklowski
Grado: Ingeniería en Inteligencia Artificial
Curso: 2026/2027
Asignatura: Razonamiento y Representación del Conocimiento
Profesor: Diego Viejo Hernando
Grupo de prácticas: 2 (viernes 19:00-21:00)
Fecha de entrega: 11 de noviembre de 2026

ÍNDICE

1. Presentación de la práctica 1

1.1. Introducción 1

1.2. Objetivos 1

2. Ejercicio 1: Sistema Experto Clásico 2

2.1. Análisis de datos de entrada 2

2.2. Funciones del sistema 2

2.2.1. Función `setObjetivo()` 2

2.2.2. Función `tomarDecision()` 4

2.3. Explicación y justificación de las reglas 5

2.3.1. Regla del estado 5

2.3.2. Reglas del control de velocidad 6

2.3.3. Reglas del frenado progresivo 9

2.4. Pruebas del sistema 10

2.5. Conclusión del ejercicio 1 11

3. Ejercicio 2: Sistema Experto Difuso 12

3.1. Definición de las variables lingüísticas 12

3.2. Definición y justificación de reglas difusas 20

3.2.1. Reglas de velocidad angular 20

3.2.2. Reglas de poca velocidad lineal 21

3.2.3. Reglas de mucha velocidad lineal 22

3.3. Definición del motor de inferencia 23

3.4. Funciones del sistema 24

3.5. Pruebas del sistema 25

3.6. Conclusión del ejercicio 2 26

4. Referencias Bibliográficas 26

1. Presentación de la práctica

1.1. Introducción

Esta documentación corresponde a la primera práctica de la asignatura de Razonamiento y Representación del Conocimiento del grado en Ingeniería en Inteligencia Artificial. La práctica consiste en diseñar un **sistema experto clásico** y un **sistema experto difuso**. En ambos casos, el sistema experto controla el movimiento de un robot en un mapa. El sistema experto debe calcular y modificar constantemente los valores de velocidad lineal y angular, en función de cuánto está desviado y de lo lejos que se encuentra del objetivo.

1.2. Objetivos

Entre los objetivos de esta práctica se destaca:

- Implementar un sistema experto clásico formulando reglas lógicas de tipo `IF-ELSE` dentro de la clase `ExpertSystem`.
- Preparar instrucciones de velocidad lineal y angular necesarias para trasladar al robot desde su posición inicial hasta el punto final del objetivo asignado.
- Priorizar que el robot se ajuste lo máximo posible al segmento que recorre, y minimizar el tiempo de recorrido.
- Desarrollar un sistema experto difuso, haciendo uso de la librería de Python `fuzzy-expert`, tratando que el robot cumpla los mismos objetivos que en el caso del sistema experto clásico.

2. Ejercicio 1: Sistema Experto Clásico

El primer ejercicio de esta práctica consiste en diseñar un sistema experto clásico utilizando únicamente las reglas básicas `IF-ELSE`. El sistema experto controla el movimiento de un robot al que se asigna un objetivo, que consiste en recorrer un circuito. El robot recibe los datos de entrada: posición del objetivo, la posición del robot en cada instante, y su orientación. A partir de estos datos, las reglas modifican la velocidad lineal y angular necesarias para que el robot alcance su objetivo.

2.1. Análisis de datos de entrada

Para que se pueda tomar una decisión adecuada, el robot necesita conocer en todo momento dónde está y hacia dónde tiene que dirigirse. Por ello, en cada instante de la ejecución, la función `tomarDecision()` recibe y procesa el objetivo y su posición actual.

El robot tiene sus límites de velocidades. La velocidad lineal puede tomar como máximo el valor de $3\ \mathrm{m/s}$, y la velocidad angular alcanza como máximo $1\ \mathrm{rad/s}$. Además, la velocidad no cambia de 0% a 100% en un instante: las aceleraciones están limitadas ($1\ \mathrm{m/s^2}$ la lineal y $0.5\ \mathrm{rad/s^2}$ la angular), así que el robot necesita distancia para frenar.

2.2. Funciones del sistema

La base del sistema experto es obtener la pose del robot y los objetivos. A continuación se explica la parte del código responsable de obtener estos datos. Los puntos del objetivo se preparan en la función `setObjetivo()`, y la decisión de movimiento (pose del robot y reglas) se toma en la función `tomarDecision()`.

2.2.1. Función `setObjetivo()`

La idea es trabajar con una lista que contiene las coordenadas obtenidas en esta función. El primer objetivo estará en la primera posición de la lista. El programa distingue dos tipos de objetivo con los cuales el robot puede enfrentarse:

- **Tipo 1 (recta):** se obtienen dos coordenadas: el punto inicial del objetivo, y el punto final. Ambos puntos se añaden a la lista de objetivos definida previamente:

```
if self.objetivoActual.getType()==1:
    self.lista_puntos.append(objetivo.getInicio())
    self.lista_puntos.append(objetivo.getFin())
```

- **Tipo 2 (triángulo):** se obtienen tres puntos: el punto inicial, el punto final, y el tercer punto es el vértice del triángulo.

```
else:
    inicio=objetivo.getInicio()
    medio=objetivo.getMedio()
    fin=objetivo.getFin()
```

Antes de añadir los tres puntos a la lista de objetivos, se calcula un punto "extra" para garantizar que el robot no va a atravesar el área del triángulo. Primero de todo se calcula el punto medio de la recta entre el inicio y el final del objetivo:

```
punto_medio=((inicio[0]+fin[0])/2.0,(inicio[1]+ fin[1])/2.0)
```

A continuación, se calcula el segmento que une el punto medio calculado previamente y el vértice del triángulo. Además, se calcula su distancia euclídea. Estos cálculos sirven para obtener un vector unitario de la dirección de la recta, para aplicar un margen de seguridad que impedirá al robot atravesar el área del triángulo.

```
vector_medio=(medio[0]-punto_medio[0],medio[1]-punto_medio[1])
distancia_medio=math.sqrt(vector_medio[0]**2+vector_medio[1]**2)
margen_seguridad=0.6
```

Se obtienen las coordenadas del punto que va a rodear el robot, aplicado ya el margen de seguridad. Se obtiene multiplicando el vector unitario (calculado previamente) por el margen de seguridad.

```
punto_rodear=(medio[0]+(vector_medio[0]/distancia_medio)*
             margen_seguridad,medio[1]+
             (vector_medio[1]/distancia_medio)*
             margen_seguridad)
```

Finalmente, se añaden los puntos calculados a la lista con la que trabajará el sistema.

```
self.lista_puntos.append(inicio)
self.lista_puntos.append(punto_rodear)
self.lista_puntos.append(fin)
```

2.2.2. Función `tomarDecision()`

Teniendo las coordenadas de los objetivos parciales en la lista, el siguiente paso es obtener la pose del robot, y definir las reglas que controlan las velocidades del robot en cada instante. A continuación se explica la parte del código responsable de obtener la pose del robot, y las reglas se explicarán en un apartado dedicado únicamente a ellas más adelante.

Antes de empezar se comprueba que la lista de puntos existe para evitar errores de ejecución. Si la lista no existe, o está vacía, entonces el robot se detiene por completo:

```
if not hasattr(self,'lista_puntos') or not self.lista_puntos:
    return (0.0,0.0)
```

Cuando ya se controla el posible error de ejecución, el siguiente paso es obtener la posición del robot:

```
x_robot=poseRobot[0]
y_robot=poseRobot[1]
angulo_robot=math.radians(poseRobot[2])
```

A continuación se obtienen las coordenadas del objetivo actual al que se va a dirigir el robot. Además, para saber cuánto recorrido le queda al robot hasta alcanzar su destino, se calcula el vector de distancia, y la distancia euclídea desde la posición actual, hasta el objetivo.

```
x_objetivo,y_objetivo=self.lista_puntos[0]
```

```
vector_distancia=(x_objetivo-x_robot,y_objetivo-y_robot)
distancia=math.sqrt(vector_distancia[0]**2
                    +vector_distancia[1]**2)
```

Para que el robot sepa cómo alinearse con su objetivo actual, sabiendo que en cada instante recibe su orientación, se calcula el ángulo de referencia. Es el ángulo deseado que el robot debe tomar para alcanzar el objetivo.

```
angulo_referencia=math.atan2(vector_distancia[1],
                             vector_distancia[0])
```

Finalmente, se calcula la diferencia entre el ángulo de referencia y el ángulo del robot para poder determinar qué giro debe realizar el robot para enfrentarse con su objetivo.

```
dif_angulo=angulo_referencia-angulo_robot
error_angulo=math.atan2(math.sin(dif_angulo),math.cos(dif_angulo))
```

Con toda la información que se ha obtenido y calculado, el sistema experto puede controlar las velocidades del robot mediante unas reglas `IF-ELSE`.

2.3. Explicación y justificación de las reglas

En este apartado se explican las reglas creadas para controlar el movimiento del robot por el mapa. Las reglas de esta implementación se dividen en tres tipos:

- Regla del estado.
- Reglas de control de la velocidad.
- Reglas de frenado progresivo.

A continuación se explica cada tipo de regla detalladamente.

2.3.1. Regla del estado

Como se ha mencionado previamente, el robot trabaja con una lista de puntos que contiene sus siguientes objetivos. Cuando el robot alcance su objetivo actual (se considera alcanzado cuando la distancia al objetivo es menor que un umbral establecido), es necesario eliminar de la lista el objetivo cumplido, para que pueda centrarse en el siguiente. La regla del estado implementada:

```
if distancia < self.umbral:
    self.lista_puntos.pop(0)

    if not self.lista_puntos:
        return (0.0,0.0)
```

Cuando no quedan más objetivos en la lista, el sistema experto para al robot (devolviendo velocidades iguales a 0.0 ambas). Si hay más puntos en la lista, se elimina el punto visitado y las reglas de control de velocidad se encargan de ajustar sus valores.

2.3.2. Reglas del control de velocidad

Este bloque de reglas se encarga de modificar las velocidades lineal y angular. A partir del error del ángulo se modifican las velocidades. Las reglas tienen la siguiente estructura general:

```
if abs(error_angulo) < umbral de error:
    v_lineal = valor de velocidad lineal
    if error_angulo > 0:
        v_angular = valor de velocidad angular
    else:
        v_angular = -valor de velocidad angular
```

La condición `abs(error_angulo)` evalúa cómo de desviado está el robot de su objetivo. Si está muy desviado, la velocidad lineal debe ser pequeña (o nula), y la velocidad angular debe ser grande. De manera opuesta, si el robot no está desviado, la velocidad lineal será grande, y la velocidad angular será pequeña (o nula). Las reglas de control de velocidades se han diseñado de manera progresiva: partiendo de una desviación pequeña y acabando en la desviación completa, y mantienen las velocidades siempre dentro del rango permitido (para velocidad lineal: $[0, 3]\,\mathrm{m/s}$, para velocidad angular: $[-1, 1]\,\mathrm{rad/s}$).

Para no explicar todas las reglas (un bloque largo de secuencias `IF-ELSE` anidadas), se explicarán tres reglas: La regla cuando el robot no está desviado (tipo 1), una regla del medio cuando el robot está bastante desviado (tipo 2), y la regla cuando el robot está totalmente desviado (tipo 3). Las reglas que faltan por explicar pertenecen al tipo 2, y lo único que cambia son los rangos del error del ángulo y los valores de velocidades que devolverá el sistema experto.

- **Tipo 1:** desviación poca/nula:

```
if abs(error_angulo) < 0.02:
    v_lineal = 3.0
    v_angular = 0.0
```

Cuando el valor absoluto del error del ángulo (`abs(error_angulo)`) es menor que 0.02, se considera que el robot está alineado. Cuando el robot no está desviado, la velocidad lineal será máxima ($3\,\mathrm{m/s}$), y la velocidad angular será nula ($0.0\,\mathrm{rad/s}$).

- **Tipo 2:** desviación media:

```
elif abs(error_angulo) < 0.40:
    v_lineal = 0.7
    if error_angulo > 0:
        v_angular = 0.55
    else:
        v_angular = -0.55
```

La regla del medio sigue la estructura general explicada al principio. En este caso, el umbral es 0.40. Como el error de ángulo indica que el robot está bastante desviado, es necesario modificar las velocidades. La velocidad lineal se modifica a $0.7\ \mathrm{m/s}$, y la velocidad angular depende del signo del error.

- **Error del ángulo $>$ 0:** indica que el objetivo se encuentra a la izquierda del robot. Se modifica la velocidad angular a $0.55\ \mathrm{rad/s}$.

- **Error del ángulo $<$ 0:** indica que el objetivo se encuentra a la derecha del robot. Se modifica la velocidad angular a $-0.55\ \mathrm{rad/s}$.

El control del giro (comprobando el signo) previene que el robot dé una vuelta entera innecesariamente. La explicación de la decisión del giro se representa mejor en la siguiente figura:

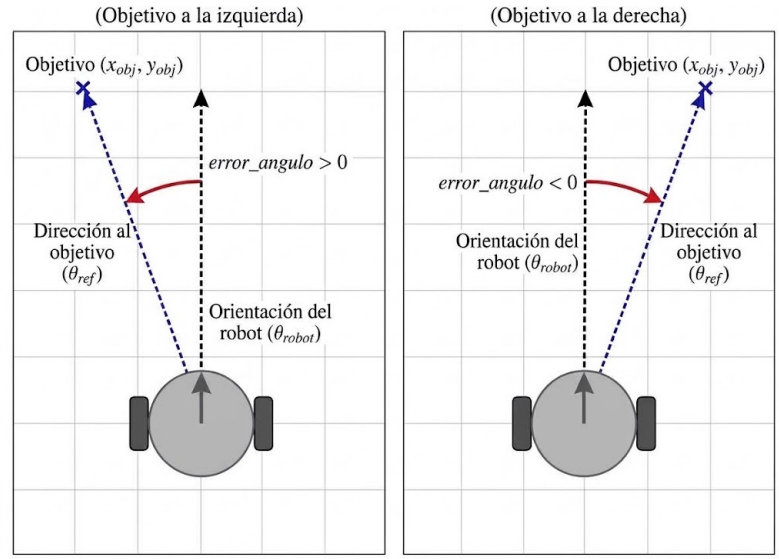


Figura 1: *Representación gráfica de elección del giro.* ***Fuente:*** *Generada con Gemini.*

- **Tipo 3:** Desviación total:

```
else:
    v_lineal = 0.0
    if error_angulo > 0:
        v_angular = 0.9
    else:
        v_angular = -0.9
```

Cuando no se ejecuta ninguna de las reglas anteriores, que corresponden al error del ángulo de 0.02 a 0.85 (radianes), se considera que el robot está completamente desviado. Por ello, la última regla modifica la velocidad lineal a 0.0 (lo para completamente) y modifica la velocidad angular a ± 0.9 (dependiendo del sentido del giro).

Para completar la explicación, la Tabla 1 recoge todos los tramos de error del sistema clásico: el primero es el tipo 1, el último es el tipo 3 y el resto son del tipo 2. El signo de la velocidad angular es siempre el del error.

Tramo del error (rad)	**Velocidad lineal (m/s)**	**Velocidad angular (rad/s)**
$\lvert e\rvert < 0.02$	3.0	0.0
$0.02 \leq \lvert e\rvert < 0.10$	2.6	± 0.15
$0.10 \leq \lvert e\rvert < 0.15$	1.7	± 0.20
$0.15 \leq \lvert e\rvert < 0.20$	1.2	± 0.31
$0.20 \leq \lvert e\rvert < 0.25$	0.9	± 0.44
$0.25 \leq \lvert e\rvert < 0.40$	0.7	± 0.55
$0.40 \leq \lvert e\rvert < 0.60$	0.5	± 0.70
$0.60 \leq \lvert e\rvert < 0.85$	0.2	± 0.80
$\lvert e\rvert \geq 0.85$	0.0	± 0.90

Tabla 1: *Reglas de control de velocidad del sistema clásico (*e *es* `error_angulo`)

2.3.3. Reglas del frenado progresivo

Para conseguir que el robot se pase del punto del objetivo actual lo menos posible, y que siga una trayectoria adecuada, se han diseñado unas reglas de frenado cuando se aproxima al objetivo.

Las reglas aparecen dentro del método `tomarDecision()` y tienen la siguiente estructura general:

```
elif distancia < umbral * constante:
    if v_lineal > valor de velocidad lineal anterior:
        v_lineal = valor de velocidad lineal nuevo
```

Ejemplo de una regla con los valores sustituidos:

```
elif distancia < self.umbral * 8:
    if v_lineal > 1.0:
        v_lineal = 1.0
```

Las reglas de frenado utilizan la variable `distancia` (distancia euclídea al objetivo actual) comparada con múltiplos de un umbral fijo (0.4 m). Se implementan reglas con las siguientes relaciones de distancias:

- Distancia mayor a 4.0 m, $(\text{umbral} * 10)$: No se activa ninguna restricción de frenado, se mantiene la velocidad lineal igual a la establecida por el error del ángulo.
- Distancia menor a 4.0 m: se modifica la velocidad lineal a 2.0 m/s.
- Distancia menor a 3.20 m, $(\text{umbral} * 8)$: se modifica la velocidad lineal a 1.0 m/s.
- Distancia menor a 1.20 m, $(\text{umbral} * 3)$: se modifica la velocidad lineal a 0.5 m/s.

2.4. Pruebas del sistema

En este apartado de la documentación se mostrará un ejemplo de ejecución del programa, mostrando la trayectoria recorrida por el robot. Se han probado diferentes recorridos:

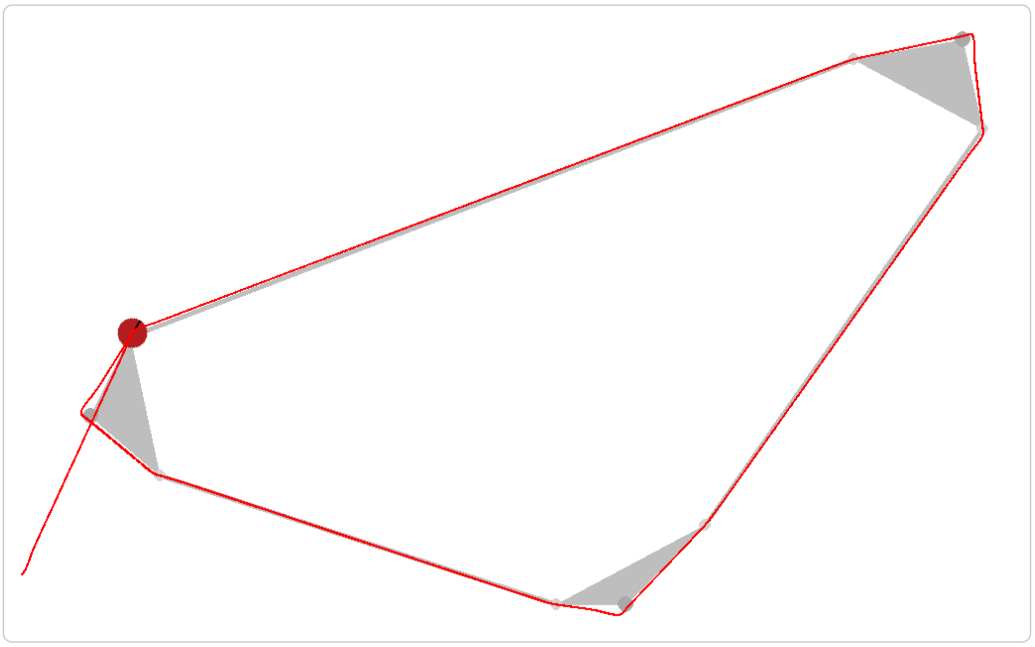


Figura 2: *Trayectoria del robot en el circuito de pruebas por defecto (recorrido 1).*

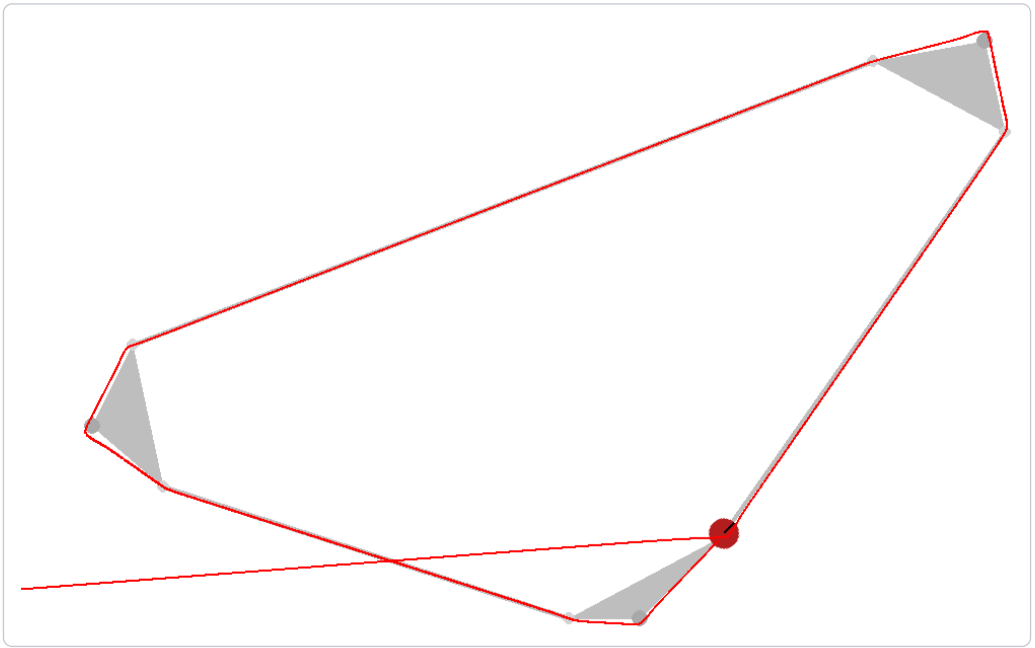


Figura 3: *Prueba del sistema en el recorrido 1b.*

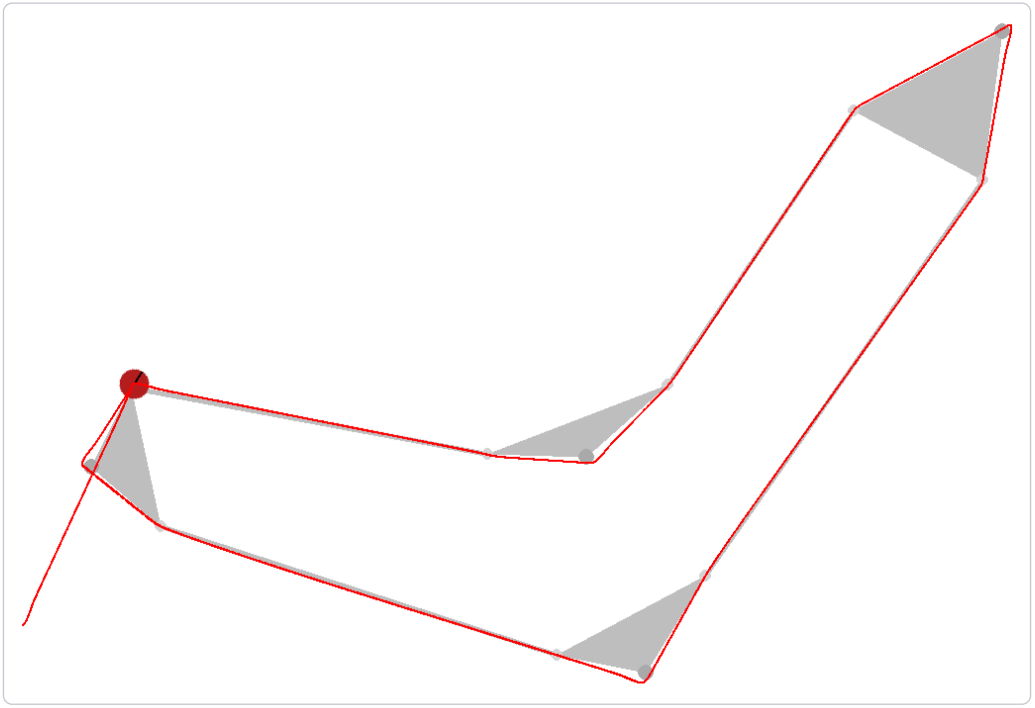


Figura 4: *Prueba del sistema en el recorrido 2.*

2.5. Conclusión del ejercicio 1

Las tres figuras muestran un ejemplo de ejecución del programa. El camino que recorre el robot está marcado en rojo. En los tres circuitos el robot sigue el camino definido por el mapa con muy poca desviación. Gracias al punto de rodear no invade ningún triángulo, y el trayecto recorrido es muy cercano al camino más corto que los evita.

Además, durante la ejecución se puede observar que el robot cambia de velocidad y lleva a cabo los giros para corregir la trayectoria. Debido al frenado progresivo se consigue buena precisión pero, a cambio, el robot es penalizado en una parte de los tests de tiempo.

En las diferentes pruebas del sistema acabado, la puntuación para el recorrido 1 varía entre 18 y 20, y para el recorrido 2 entre 30 y 35 puntos. Dependiendo de la máquina la puntuación será mayor o menor, ya que el tiempo se mide en tiempo real en cada ejecución. Desde el script vacío, hasta el código terminado se han ido modificando las reglas (añadiendo o borrando, modificando los umbrales en cada regla), para conseguir la mejor combinación entre la precisión del recorrido y una velocidad adecuada. Sin embargo, si se desea obtener buena precisión del camino recorrido, habrá que sacrificar unos instantes de tiempo para que el robot se alinee con su objetivo. A pesar de todo, se ha intentado encontrar el mejor ajuste entre el tiempo y la trayectoria.

3. Ejercicio 2: Sistema Experto Difuso

El segundo ejercicio de esta práctica consiste en diseñar un sistema experto difuso, utilizando la librería `fuzzy-expert` [4]. Su objetivo es exactamente el mismo que en el sistema experto clásico, controlar el robot para alcanzar un objetivo. A diferencia de la lógica clásica, el sistema experto difuso modela el conocimiento a través de grados de pertenencia. Esto permite que el sistema evalúe las entradas (como la distancia y el error angular) en términos lingüísticos (por ejemplo, "cerca", "lejos", "poco desviado"), y realice una transición de velocidades más suave que la del sistema clásico.

3.1. Definición de las variables lingüísticas

El primer paso en el diseño del sistema experto es la fuzzificación (*fuzzification*). Consiste en traducir las magnitudes (como la distancia o el error angular) a variables lingüísticas.

La librería `fuzzy-expert` define los conjuntos difusos en diferentes formas geométricas. Este sistema experto se basa en tres:

- **Triangular Membership Function** (`trimf`): Crea un conjunto en forma de triángulo definido por tres puntos (inicio, pico máximo y fin). La pertenencia es máxima en el valor central y decae linealmente hacia los extremos.

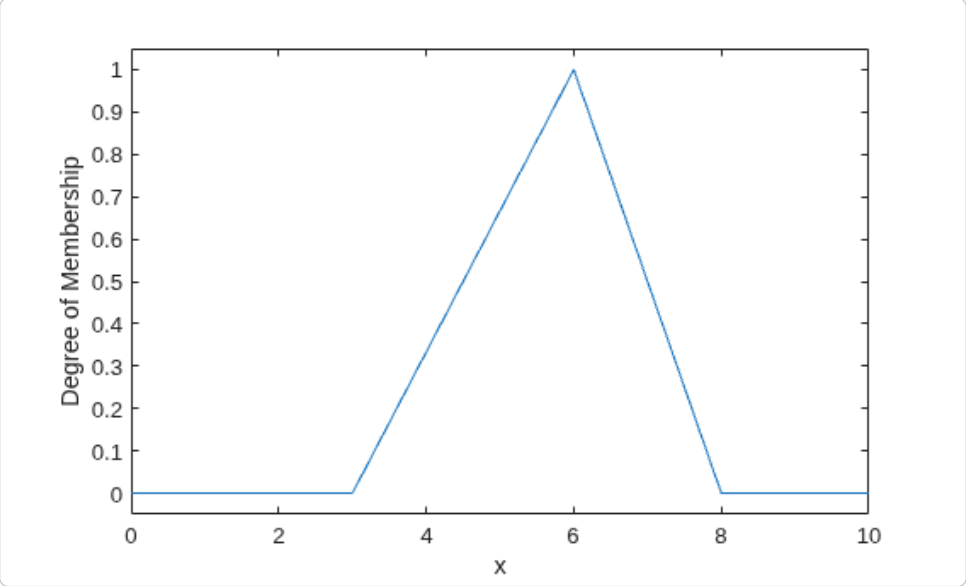


Figura 5: *Representación de la función* `trimf` .
Fuente*: MathWorks [1].*

- **Z-shaped Membership Function** (`zmf`): Crea una curva suave en forma de "Z" que comienza con una pertenencia total (1) en los valores más bajos y cae gradualmente hasta cero.

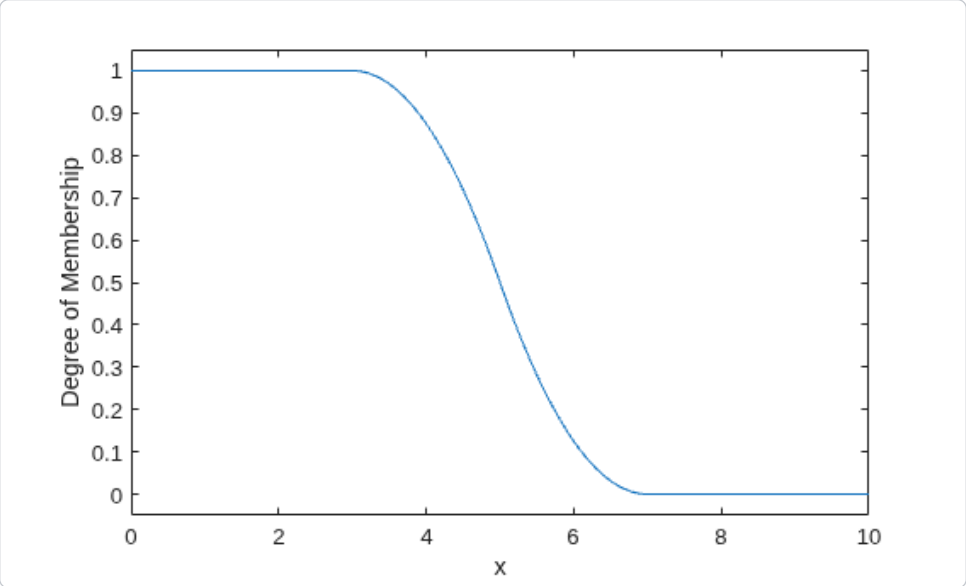


Figura 6: *Representación de la función* `zmf` *.*
Fuente*: MathWorks [2].*

- **S-shaped Membership Function** (`smf`): Crea una curva suave en forma de "S" que empieza en cero y sube gradualmente hasta la pertenencia total (1) en los valores más altos.

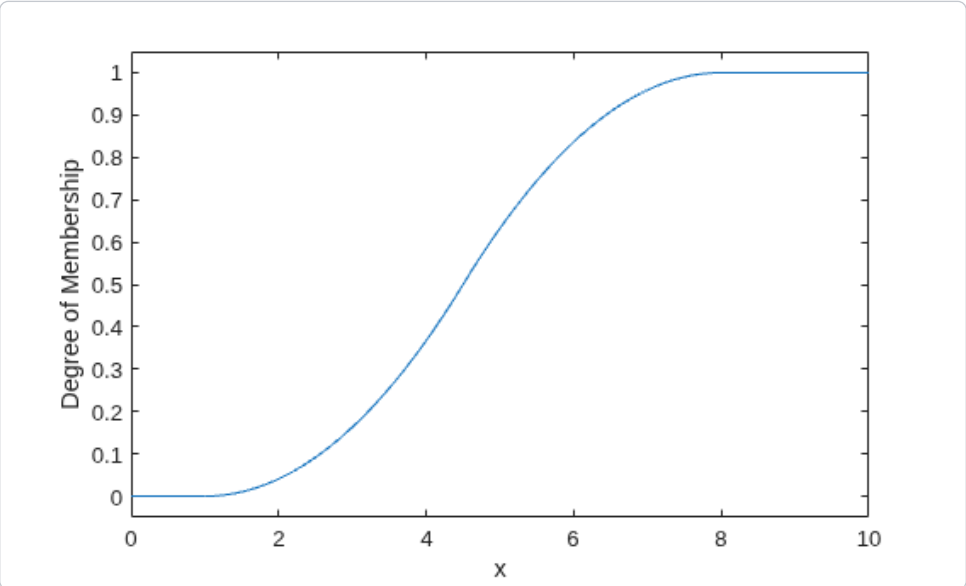


Figura 7: *Representación de la función* `smf` *.*
Fuente*: MathWorks [3].*

Las variables se dividen en variables de entrada (error del ángulo y distancia), y en variables de salida (velocidad lineal y velocidad angular). Las variables se definen en un diccionario (`self.variables`) dentro de la clase `FuzzySystem`.

- **Variable de entrada: error del ángulo**

```
'error_angulo': FuzzyVariable(
    universe_range=(-math.pi, math.pi),
    terms={
        'muy_derecha':('zmf', -1.0, -0.4),
        'derecha':('trimf', -1.0, -0.4, 0.0),
        'recto':('trimf', -0.4, 0.0, 0.4),
        'izquierda':('trimf', 0.0, 0.4, 1.0),
        'muy_izquierda':('smf', 0.4, 1.0),
    }
)
```

La primera variable representa el error del ángulo. Su universo de discurso es desde $-\pi$ hasta π radianes. El dominio se distribuye en 5 términos:

- **`muy_derecha`** y **`muy_izquierda`**: Representan desviaciones críticas de la trayectoria. Son muy importantes para orientar al robot hacia su objetivo cuando está totalmente desviado.
- **`derecha`** y **`izquierda`**: Representan un error de orientación medio. Permiten realizar correcciones de giro suave para orientar el robot hacia su objetivo sin generar movimientos bruscos.
- **`recto`**: Un triángulo con pico (pertenencia máxima) en 0. Cuando el error crece, `derecha` e `izquierda` se activan a la vez y la velocidad lineal baja de forma progresiva. Evita corregir milimétricamente la trayectoria y reduce la oscilación.

- **Variable de entrada: distancia**

```
'distancia': FuzzyVariable(
    universe_range=(0,100),
    terms={
        'muy_cerca':('zmf', 0.6, 1.7),
        'cerca':('zmf', 1.3, 3.33),
        'media':('trimf', 2.64, 20.8, 26),
        'lejos':('smf', 11.9, 33.0)
    }
)
```

La segunda variable representa la distancia euclídea desde el robot hasta su objetivo en un instante dado. Su dominio se define en un intervalo de $[0, 100]$ (metros). La variable consta de 4 términos:

- `muy_cerca` : Representa la distancia muy pequeña hasta el objetivo.
- `cerca` : Representa una zona relativamente aproximada al objetivo. Su solapamiento con los términos `muy_cerca` y `media` permite ir reduciendo la velocidad con antelación.
- `media` : Cubre las distancias intermedias, en las que el robot alineado todavía puede ir a velocidad máxima. Al acercarse, su pertenencia baja y `cerca` gana más importancia, que es el comienzo del frenado.
- `lejos` : Representa una lejanía absoluta. Indica al sistema experto que el robot está lo suficiente lejos para aplicar con seguridad la velocidad lineal máxima.

- **Variable de salida: velocidad lineal**

```
'velocidad_lineal': FuzzyVariable(
    universe_range=(0,3),
    terms={
        'poca':('zmf', 0.09, 2.35),
        'media':('trimf', 1.34, 2.2, 2.9),
        'mucha':('smf', 2.95, 3.0)
    }
)
```

Esta es la primera variable de salida del sistema experto difuso. Su dominio de discurso está definido por el enunciado en un intervalo $[0, 3]$ m/s. Al estar los grados de pertenencia solapados en los términos, el sistema experto favorece una aceleración y un frenado progresivos y suaves. La variable se modela de la siguiente manera:

- `poca` : Representa el rango de velocidades de precaución y maniobra de giro. Su máximo grado de pertenencia en los valores bajos permite al robot frenar suavemente al acercarse a su objetivo.
- `media` : Actúa como puente entre la maniobra y la aceleración máxima.
- `mucha` : Permite al robot acelerar al máximo. Las reglas solo la activan con el robot alineado y lejos del objetivo.

- **Variable de salida: velocidad angular**

```
'velocidad_angular': FuzzyVariable(
    universe_range=(-1,1),
    terms={
        'giro_fuerte_derecha':('zmf', -0.8, -0.28),
        'giro_flojo_derecha':('trimf', -1.0, -0.8, 0.0),
        'recto':('trimf', -0.09, 0.0, 0.09),
        'giro_flojo_izquierda':('trimf', 0.0, 0.8, 1.0),
        'giro_fuerte_izquierda':('smf', 0.28, 0.8)
    }
)
```

  Esta variable representa la velocidad angular. Su dominio está definido en un intervalo $[-1, 1]$ rad/s. El valor negativo de la velocidad angular representa un giro a la derecha, y su valor positivo, un giro a la izquierda. La variable se distribuye en los siguientes términos:

  – `giro_fuerte_izquierda` y `giro_fuerte_derecha`: Modelan unas rotaciones muy exigentes, cuando el robot está fuertemente desviado. Aseguran que cuando ocurra un desvío crítico, el robot aplique una velocidad angular alta (algo menor que la máxima de 1 rad/s, por el efecto del centro de gravedad).

  – `giro_flojo_izquierda` y `giro_flojo_derecha`: Modelan giros suaves que permiten corregir la trayectoria de forma gradual.

  – `recto`: Representa un estado de traslación sin rotación necesaria. Impide realizar rotaciones innecesarias que empeorarían la trayectoria con oscilación.

A continuación se muestran unas figuras que representan cada variable con sus términos y sus grados de pertenencia difusos.

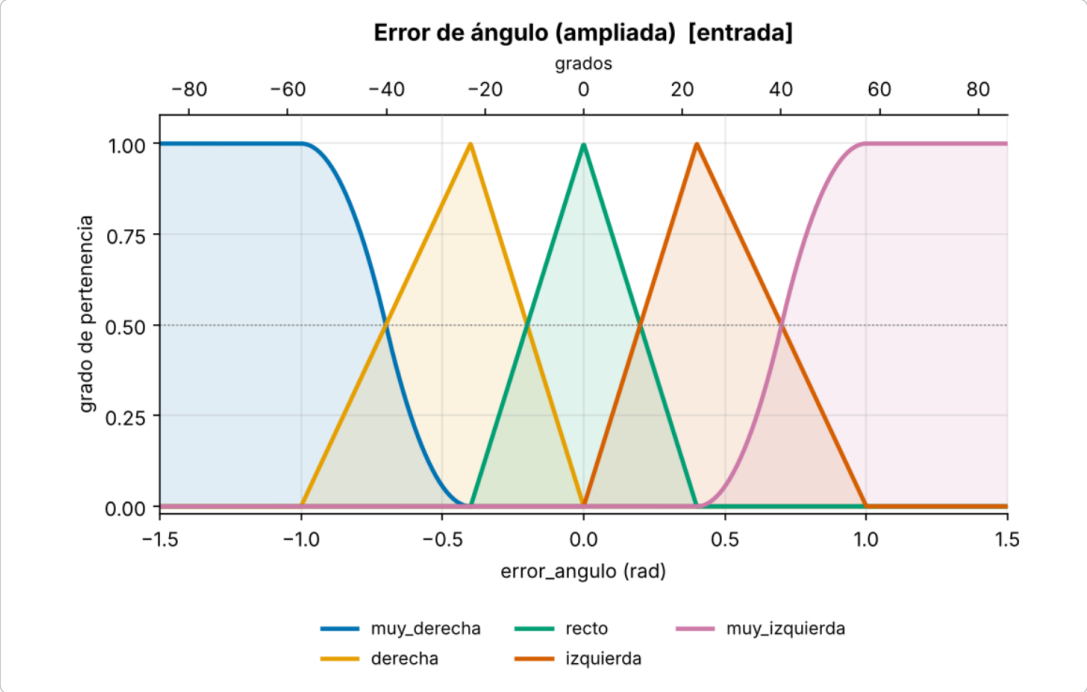


Figura 8: *Variable* `error_angulo`. ***Fuente****: Generada con Claude*

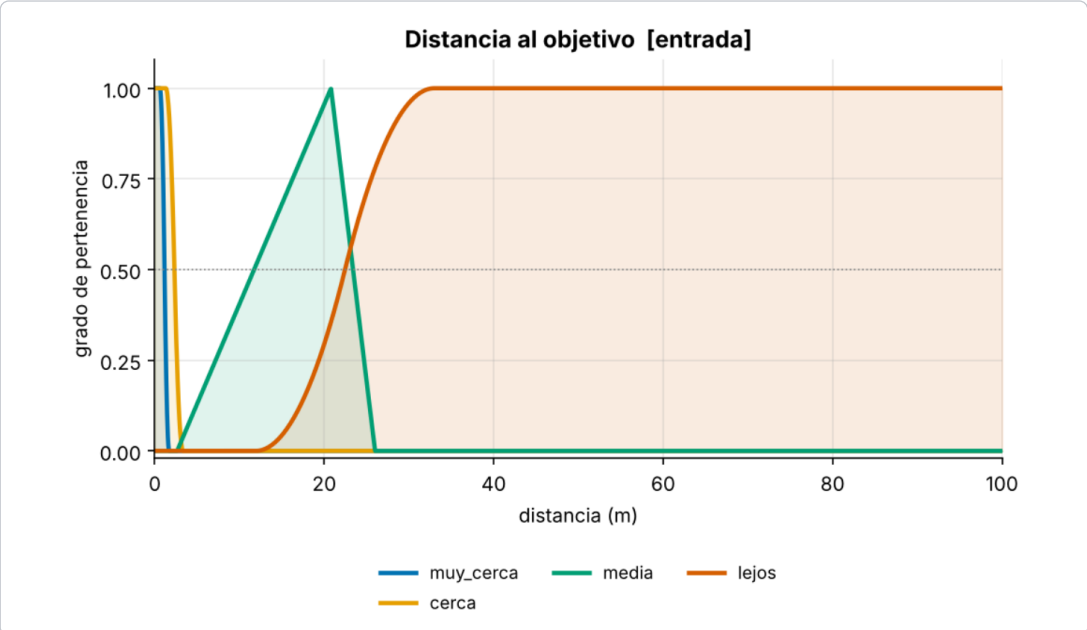


Figura 9: *Variable* `distancia`. ***Fuente****: Generada con Claude.*

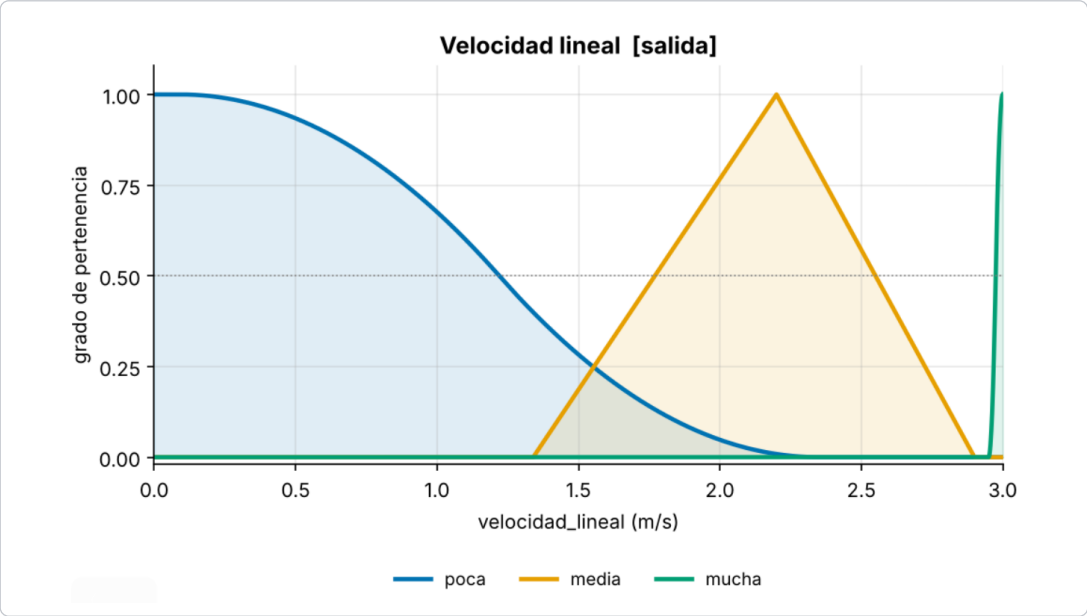


Figura 10: *Variable* `velocidad_lineal`. ***Fuente****: Generada con Claude.*

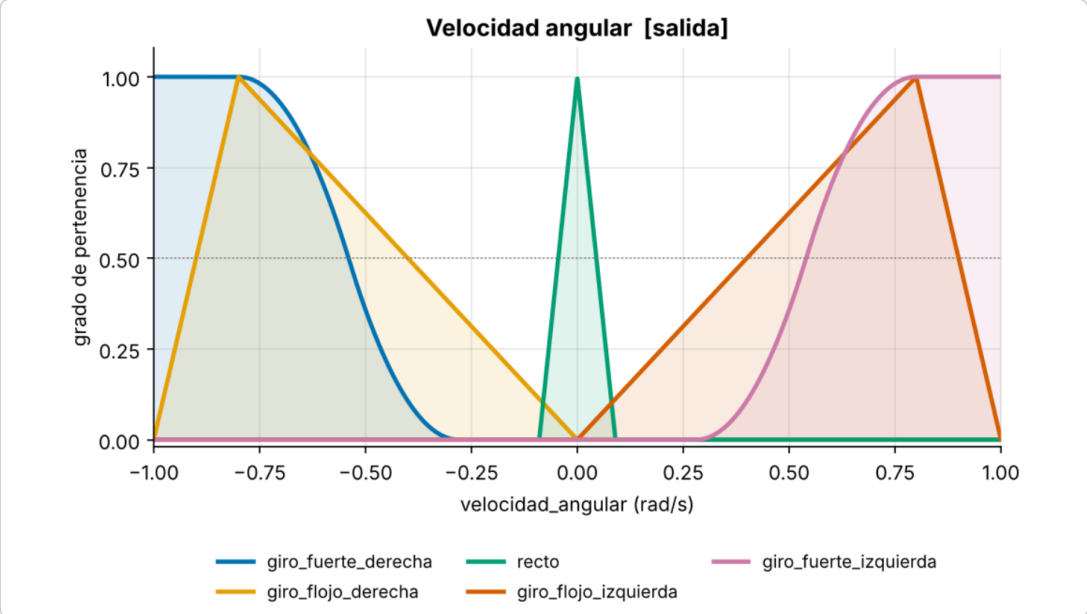


Figura 11: *Variable* `velocidad_angular`. ***Fuente****: Generada con Claude.*

3.2. Definición y justificación de reglas difusas

En este apartado se explicarán las reglas del sistema experto difuso. Las reglas se definen en una lista dentro de la clase y trabajan con las variables definidas previamente. Gracias al solapamiento de las variables lingüísticas, varias reglas pueden estar activas al mismo tiempo con distintos porcentajes de influencia. El sistema fusiona todas las decisiones, y aplica la defuzzificación para calcular un único valor numérico para cada variable de salida.

Las reglas de este sistema experto se dividen en varios grupos:

- Reglas de velocidad angular.
- Reglas de poca velocidad lineal.
- Reglas de mucha velocidad lineal.

3.2.1. Reglas de velocidad angular

El primer grupo de reglas se encarga de controlar la dirección del robot, modificando la variable de salida `velocidad_angular`, para corregir la trayectoria del robot. Todas las reglas reciben el error del ángulo y devuelven velocidad angular.

```
FuzzyRule(
    premise=[('error_angulo', 'muy_derecha')],
    consequence=[('velocidad_angular', 'giro_fuerte_derecha')]),

FuzzyRule(
    premise=[('error_angulo', 'derecha')],
    consequence=[('velocidad_angular', 'giro_flojo_derecha')]),

FuzzyRule(
    premise=[('error_angulo', 'recto')],
    consequence=[('velocidad_angular', 'recto')]),

FuzzyRule(
    premise=[('error_angulo', 'izquierda')],
    consequence=[('velocidad_angular', 'giro_flojo_izquierda')]),

FuzzyRule(
    premise=[('error_angulo', 'muy_izquierda')],
    consequence=[('velocidad_angular', 'giro_fuerte_izquierda')])
```

Explicación intuitiva para cada regla en orden:

1) Robot muy desviado a la derecha → mucha velocidad angular (realizar giro fuerte a la derecha).
2) Robot desviado un poco a la derecha → poca velocidad angular (realizar giro flojo a la derecha).
3) Robot no desviado (recto) → velocidad angular nula (no realizar giro, seguir recto)
4) Robot desviado un poco a la izquierda → poca velocidad angular (realizar giro flojo a la izquierda).
5) Robot muy desviado a la izquierda → mucha velocidad angular (realizar giro fuerte a la izquierda).

3.2.2. Reglas de poca velocidad lineal

Este segundo grupo de reglas se encarga de frenar al robot cuando necesita maniobrar. Si el sistema detecta que el robot no está mirando exactamente hacia el objetivo, fuerza la `velocidad_lineal` a ser "poca" sin importar a qué distancia esté. Esto asegura que el robot priorice el giro y no se salga de la ruta por ir demasiado rápido mientras corrige la trayectoria.

```
FuzzyRule(
    premise=[('error_angulo', 'muy_derecha')],
    consequence=[('velocidad_lineal', 'poca')]),

FuzzyRule(
    premise=[('error_angulo', 'derecha')],
    consequence=[('velocidad_lineal', 'poca')]),

FuzzyRule(
    premise=[('error_angulo', 'izquierda')],
    consequence=[('velocidad_lineal', 'poca')]),

FuzzyRule(
    premise=[('error_angulo', 'muy_izquierda')],
    consequence=[('velocidad_lineal', 'poca')])
```

Explicación intuitiva para cada regla en orden:

1) Robot muy desviado a la derecha → poca velocidad lineal.
2) Robot desviado a la derecha → poca velocidad lineal.
3) Robot desviado a la izquierda → poca velocidad lineal.
4) Robot muy desviado a la izquierda → poca velocidad lineal.

3.2.3. Reglas de mucha velocidad lineal

El último grupo de reglas controla la aceleración. Para que el robot acelere, no es suficiente solo con mirar el ángulo, sino que se tienen que cumplir dos condiciones a la vez (`AND`). El robot debe estar alineado y tener suficiente distancia por delante. De esta forma, el sistema recorta el tiempo de viaje corriendo a máxima velocidad solo en los tramos rectos donde es seguro hacerlo.

```
FuzzyRule(
    premise=[('error_angulo', 'recto'),
             ('AND', 'distancia', 'lejos')],
    consequence=[('velocidad_lineal', 'mucha')]),

FuzzyRule(
    premise=[('error_angulo', 'recto'),
             ('AND', 'distancia', 'media')],
    consequence=[('velocidad_lineal', 'mucha')]),

FuzzyRule(
    premise=[('error_angulo', 'recto'),
             ('AND', 'distancia', 'cerca')],
    consequence=[('velocidad_lineal', 'media')]),

FuzzyRule(
    premise=[('error_angulo', 'recto'),
             ('AND', 'distancia', 'muy_cerca')],
    consequence=[('velocidad_lineal', 'poca')])
```

Explicación intuitiva para cada regla en orden:

1) Robot está alineado **y** queda mucha distancia por delante → mucha velocidad lineal.
2) Robot está alineado **y** queda bastante distancia por delante → mucha velocidad lineal.
3) Robot está alineado **y** queda poca distancia por delante → velocidad lineal media.
4) Robot está alineado **y** queda muy poca distancia por delante → poca velocidad lineal.

3.3. Definición del motor de inferencia

El motor de inferencia es el núcleo lógico del sistema. A diferencia del control clásico (`IF-ELSE`), el motor difuso evalúa todas las reglas a la vez.

En cada iteración, el motor lee las variables de entrada y activa múltiples reglas a la vez con distintos porcentajes de influencia, gracias al solapamiento de los conjuntos difusos. A continuación, fusiona todas estas respuestas y aplica el proceso de defuzzificación para calcular un único valor para las velocidades del robot. Este cálculo combinado favorece que el robot navegue de forma más fluida, con cambios de velocidad graduales.

```
self.motor_inferencia = DecompositionalInference(
    and_operator='min',
    or_operator='max',
    implication_operator='Rc',
    composition_operator='max-min',
    production_link='max',
    defuzzification_operator='cog',
)
```

3.4. Funciones del sistema

El sistema experto difuso utiliza la misma estructura de funciones principales que el sistema experto clásico. La función `setObjetivo()` no sufre ninguna modificación, por lo que la explicación de su lógica y cálculos geométricos se encuentra detallada previamente en el apartado **2.2.1. Función** `setObjetivo()` de este documento.

Por su parte, la función `tomarDecision()` conserva la primera fase explicada en el apartado **2.2.2. Función** `tomarDecision()`: la obtención de la pose del robot y el cálculo de la distancia euclídea y del error angular son los mismos. La única diferencia es el valor del umbral para dar por alcanzado un objetivo; se modifica a 0.28 m (en el clásico, 0.4 m).

El cambio principal reside en la segunda mitad de la función. Se ha eliminado todo el bloque de reglas condicionales `IF-ELSE` para sustituirlo por una llamada directa al motor de inferencia difuso. El sistema simplemente inyecta los valores calculados en las variables de entrada y extrae las velocidades ya defuzzificadas.

Código de la llamada al motor de inferencia:

```
resultado, _ = self.motor_inferencia(
    variables=self.variables,
    rules=self.reglas,
    error_angulo=error_angulo,
    distancia=distancia
)
```

Código responsable de la extracción de los resultados:

```
V = resultado.get('velocidad_lineal', 0.0)
W = resultado.get('velocidad_angular', 0.0)
```

Como se puede observar, el motor difuso devuelve un diccionario con los resultados ya procesados. Mediante el proceso de defuzzificación, la librería extrae los valores de cada variable de salida. Además se llama al método `get()` con el valor `0.0` como parámetro por defecto, lo que evita un error por clave inexistente si en algun caso extraño la variable de salida no apareciera en el resultado.

3.5. Pruebas del sistema

En este apartado de la documentación se mostrará un ejemplo de ejecución del programa, mostrando la trayectoria recorrida por el robot. Se han probado diferentes recorridos:

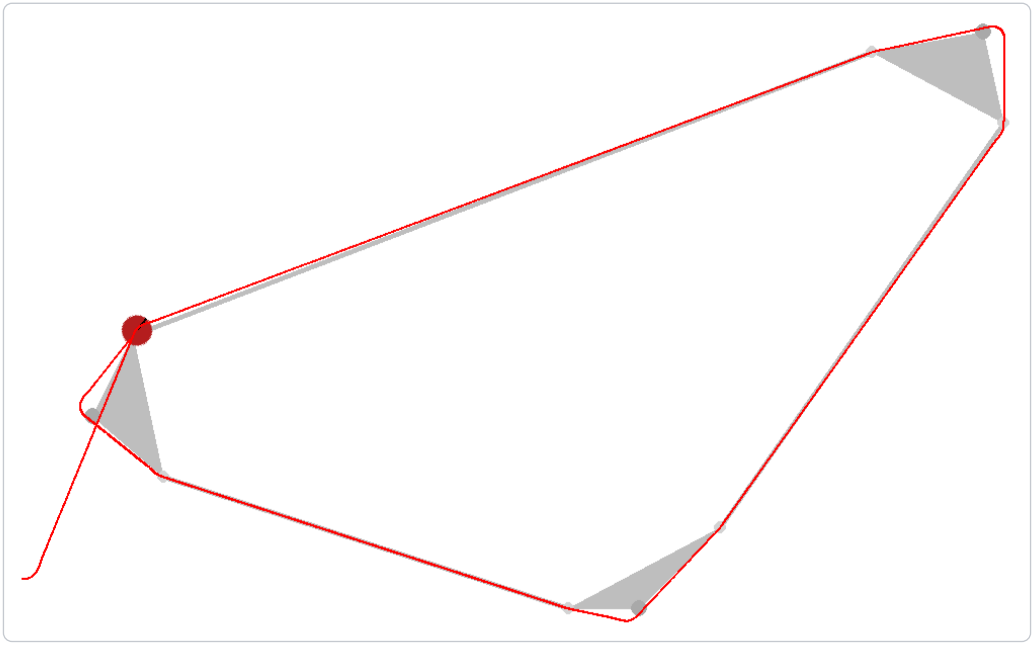


Figura 12: *Prueba del sistema en el recorrido 1.*

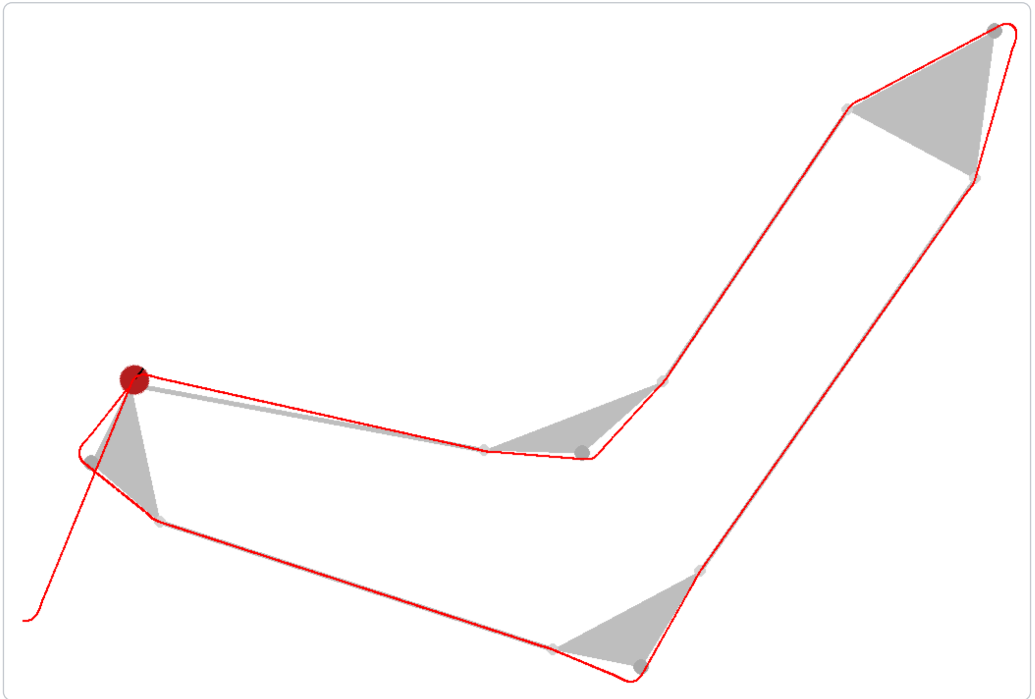


Figura 13: *Prueba del sistema en el recorrido 2.*

3.6. Conclusión del ejercicio 2

Las dos figuras muestran los dos recorridos propuestos tal y como los ha recorrido este sistema experto difuso. Como en el caso del sistema clásico, el camino recorrido por el robot está marcado en rojo. Al aplicar exactamente los mismos cálculos del punto de rodear, el sistema garantiza que el robot no invada ningún área del triángulo.

Durante la ejecución se puede observar que el robot alcanza los objetivos propuestos, manejando las velocidades y frenando cuando es necesario. Se ha conseguido muy buena precisión del recorrido, pero, como en el ejercicio anterior (debido al frenado), el robot es penalizado en una parte de las pruebas de tiempo.

En diferentes ejecuciones, las puntuaciones para el recorrido 1 varían entre 20 y 25 puntos, y para el recorrido 2, alrededor de los 40. Igual que en el ejercicio 1, la puntuación depende del ordenador. Durante todo el proceso de diseño del sistema experto difuso se han ido ajustando los términos después de cada prueba para mejorar el rendimiento del robot, así como modificar o eliminar reglas redundantes.

4. Referencias Bibliográficas

- **[1] MathWorks**, *trimf - Triangular membership function*. Fuzzy Logic Toolbox.
- **[2] MathWorks**, *zmf - Z-shaped Membership Function*. Fuzzy Logic Toolbox.
- **[3] MathWorks**, *smf - S-shaped Membership Function*. Fuzzy Logic Toolbox.
- **[4] fuzzy-expert**, *fuzzy-expert*, librería de Python para sistemas expertos difusos.

NOTA: *Para elaborar este documento se han utilizado herramientas de inteligencia artificial para la mejora de la redacción y generación de las gráficas.*